



空间数据库原理与应用

第十一讲：空间数据库开发（一）

任课教师：李晓明

建筑与城市规划学院 城市空间信息工程系

其中部分图片来自互联网和同行专家

目录

CONTENTS

01

数据库开发概述

02

嵌入式SQL

03

动态SQL

目录

CONTENTS

1

数据库开发

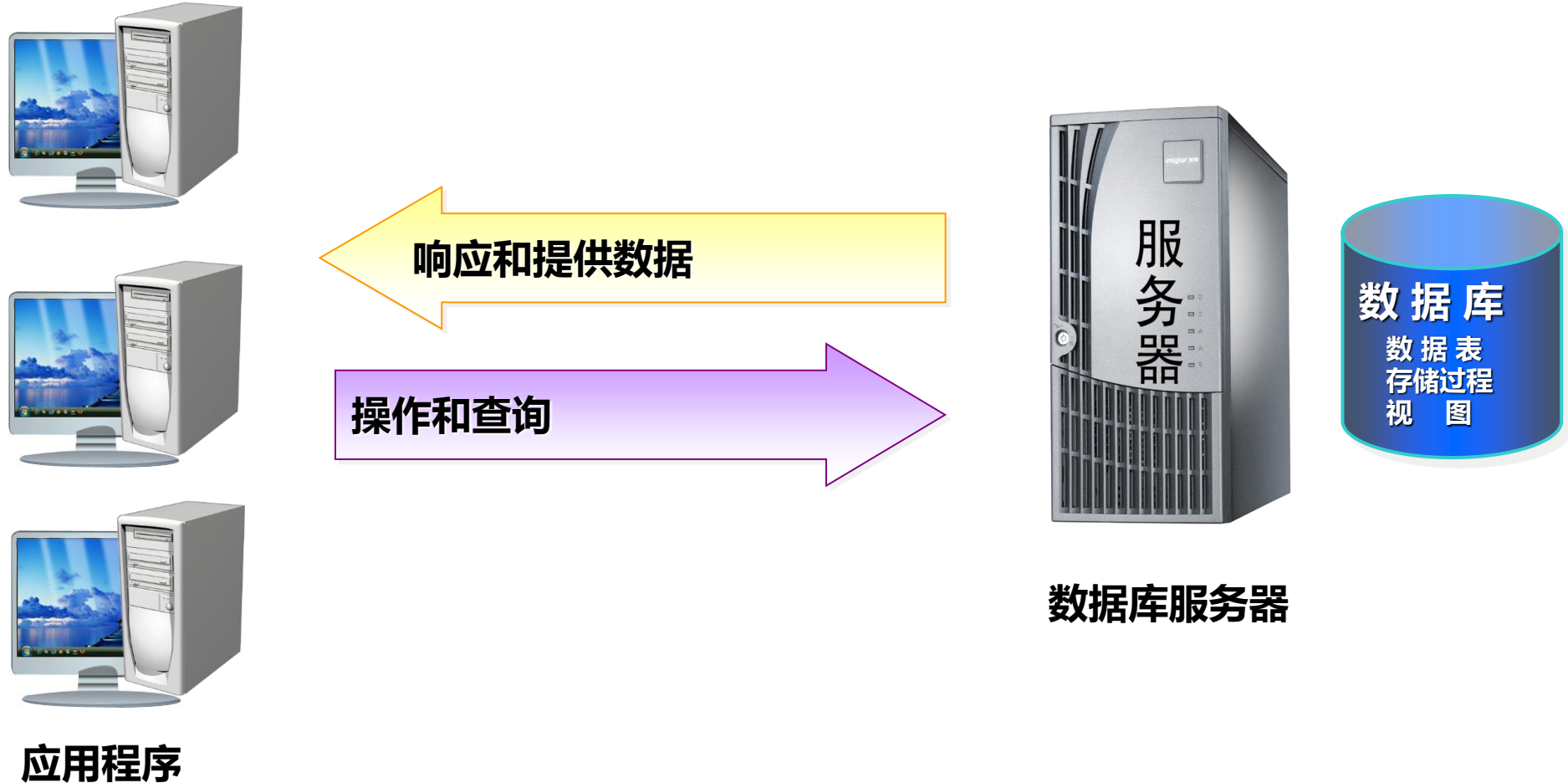
1.1 数据库开发概述

1.2 数据库访问技术

1.3 数据库开发实例

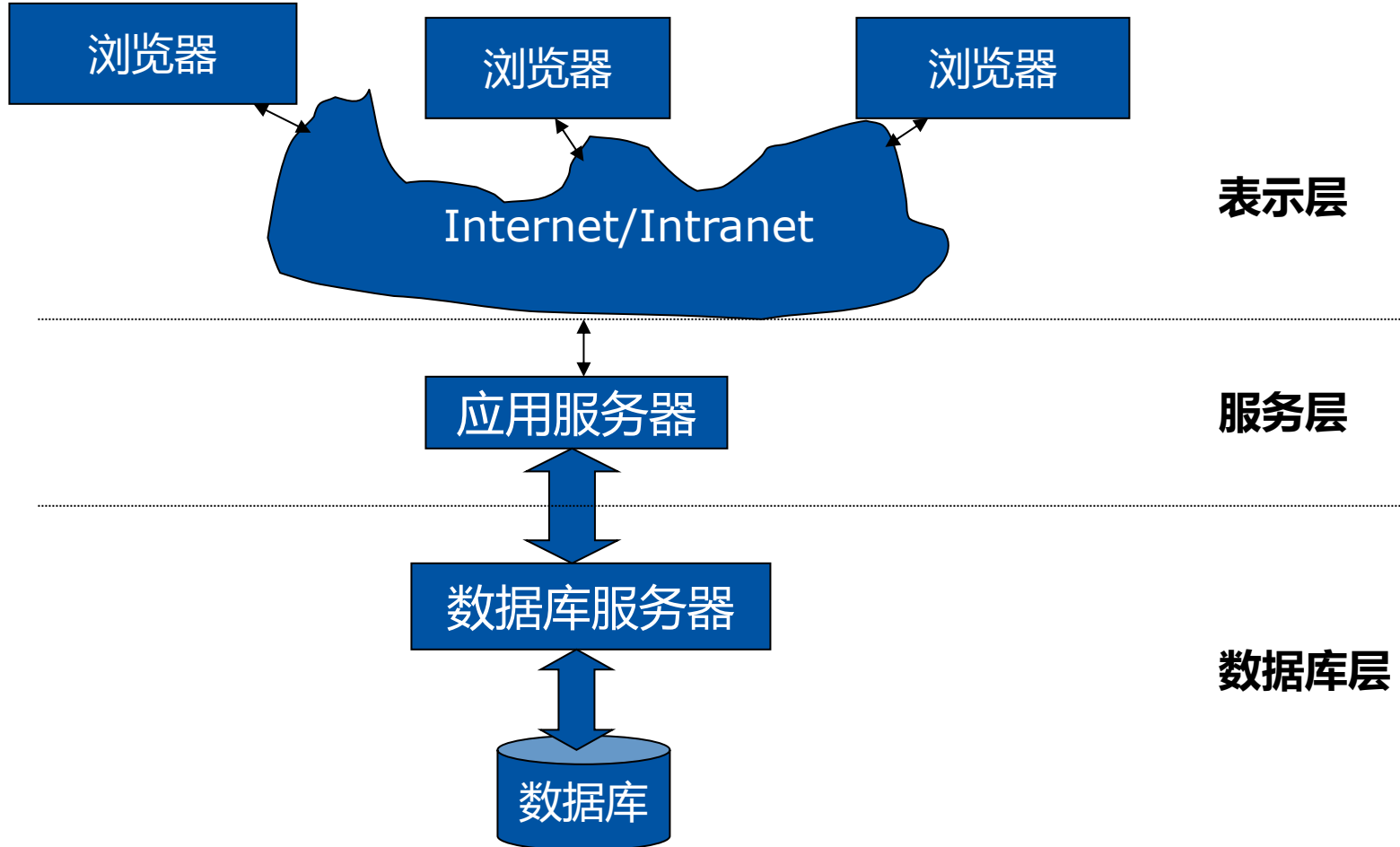
1.1 数据库开发概述

数据库与应用程序和服务器的关系 (C/S结构)



1.1 数据库开发概述

数据库与应用程序和服务器的关系 (B/S结构)



应用服务器:

- 用途:
 - 提供WWW服务, 支持数据库访问技术 (ODBC、JDBC、OLE DB、ADO) 管理与配置应用系统;
- 可以安装以下Web服务:
 - IIS
 - Apache
 - Netscape—NES
 - Java Web Server
 - Application Server
 - WebSphere
- 使用以下技术与数据库相连
 - CGI、ASP、PHP、JSP

1.1 数据库开发概述

数据库编程概述

SNO	SN	AGE	DEPT
S1	赵亦	17	计算机
S2	钱尔	18	信息
S3	孙珊	20	信息
S4	李思	21	自动化

D		
DEPT		MN
计算机		刘伟
信息		王平
自动化		刘伟



数据访问
方式

学生基本情况

S_id:

Sn:

Sex:

Age:

Dept:

电话号码:

备注:

第一个 (F)
前一个 (P)
下一个 (N)
最后一个 (L)

程序界面
设计

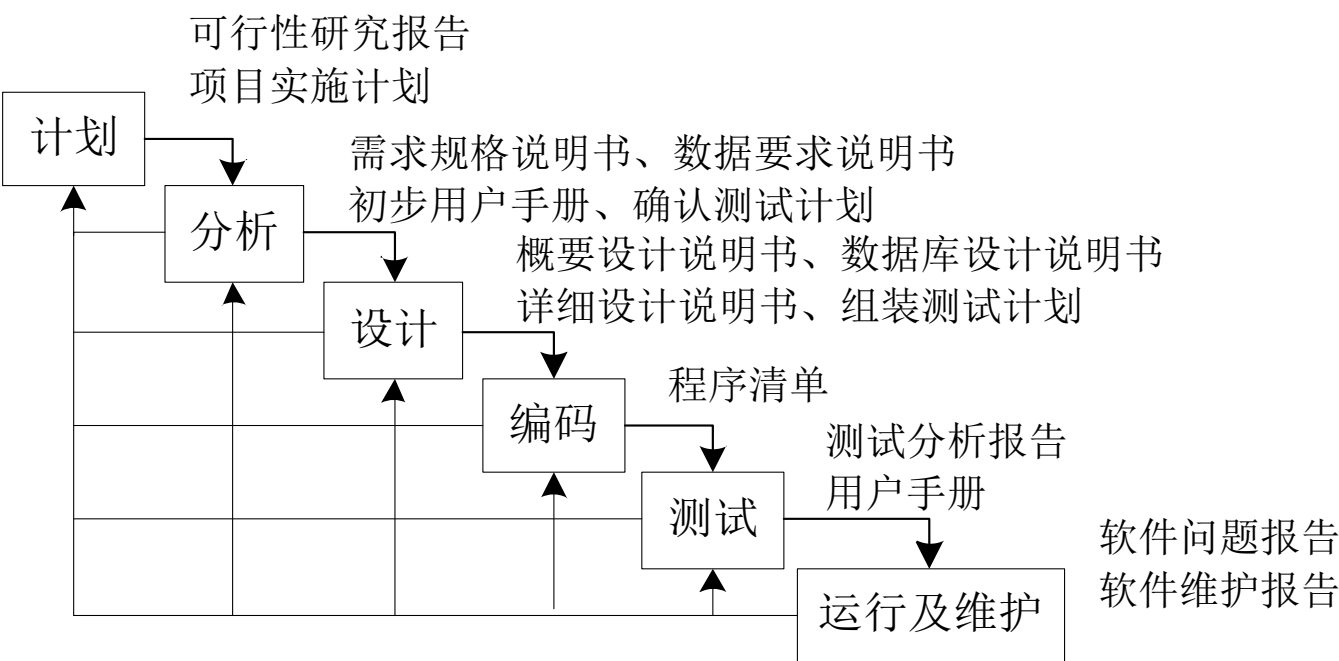
1.1 数据库开发概述

❖ 数据库应用系统（DBAS）：

- 为了完成某一个特定的任务，把与该任务相关的数据以某种数据模型进行存储，并围绕这一目标开发的应用程序。通常把这些**数据、数据模型以及应用程序整体**称作为一个数据库应用系统。

❖ 数据库应用系统的开发过程的6个阶段

1. 计划
2. 分析
3. 设计
4. 编码
5. 测试
6. 运行及维护

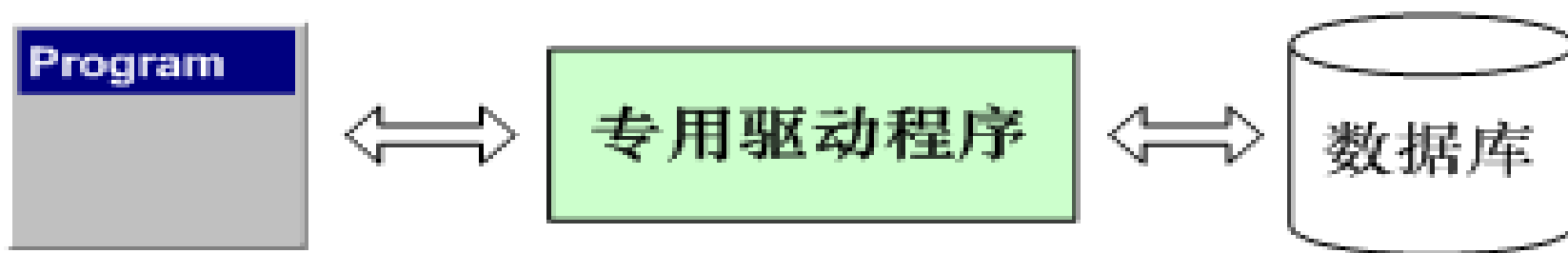


数据库应用系统的开发过程

1.2 数据库访问技术

数据访问方式的变迁

- 1. 直接访问数据库

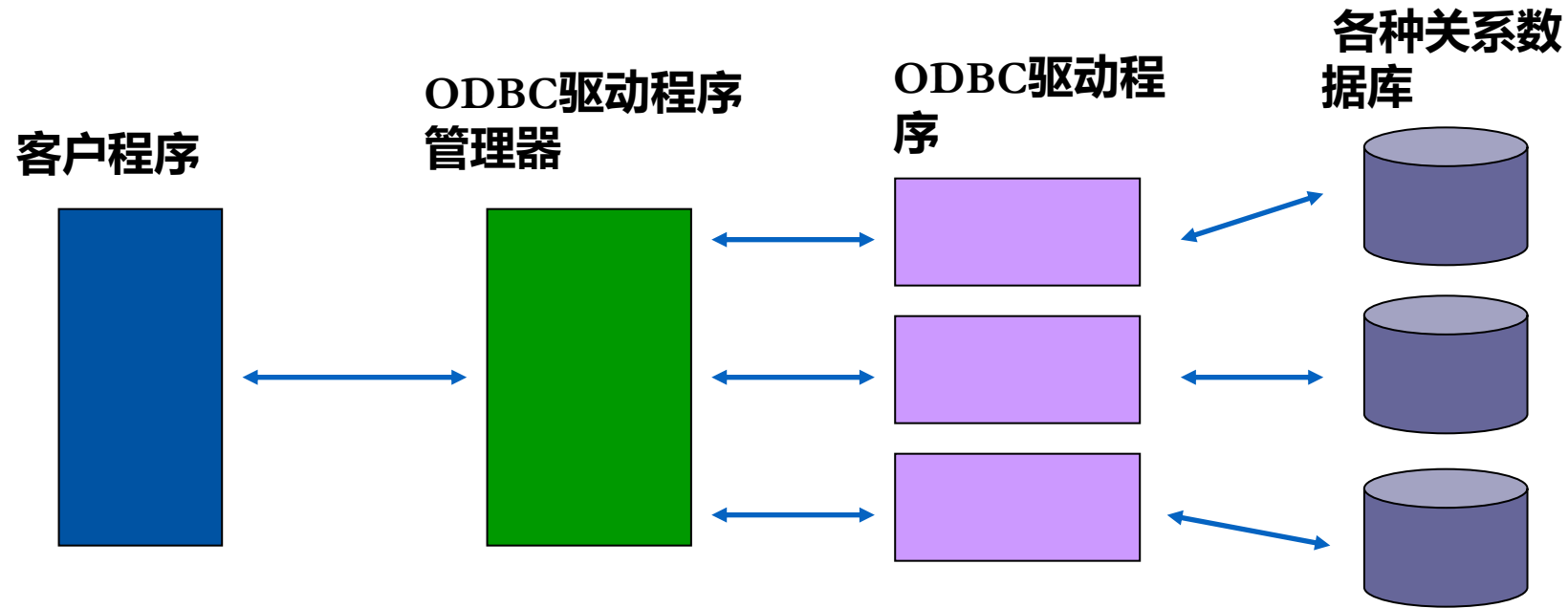


使用数据库系统所提供的专用开发工具(如嵌入式SQL语言)，开发的应用程序只能运行在特定的数据库系统环境下，**适应性和可移植性比较差。**

1.2 数据库访问技术

数据访问方式的变迁

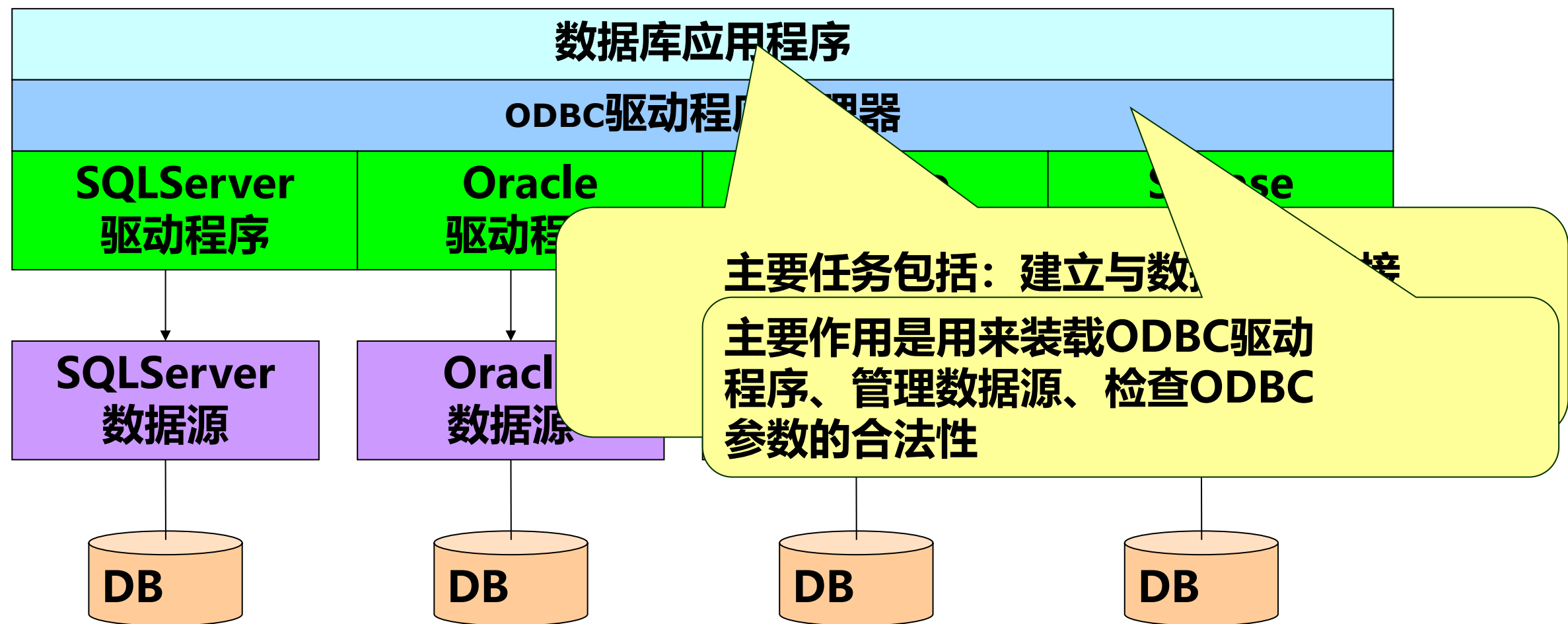
- 2. ODBC (Open Database Connectivity 开放数据库连接)



ODBC是上个世纪八十年代末九十年代初出现的技术，它为编写关系数据库的客户软件提供了一种统一的接口。

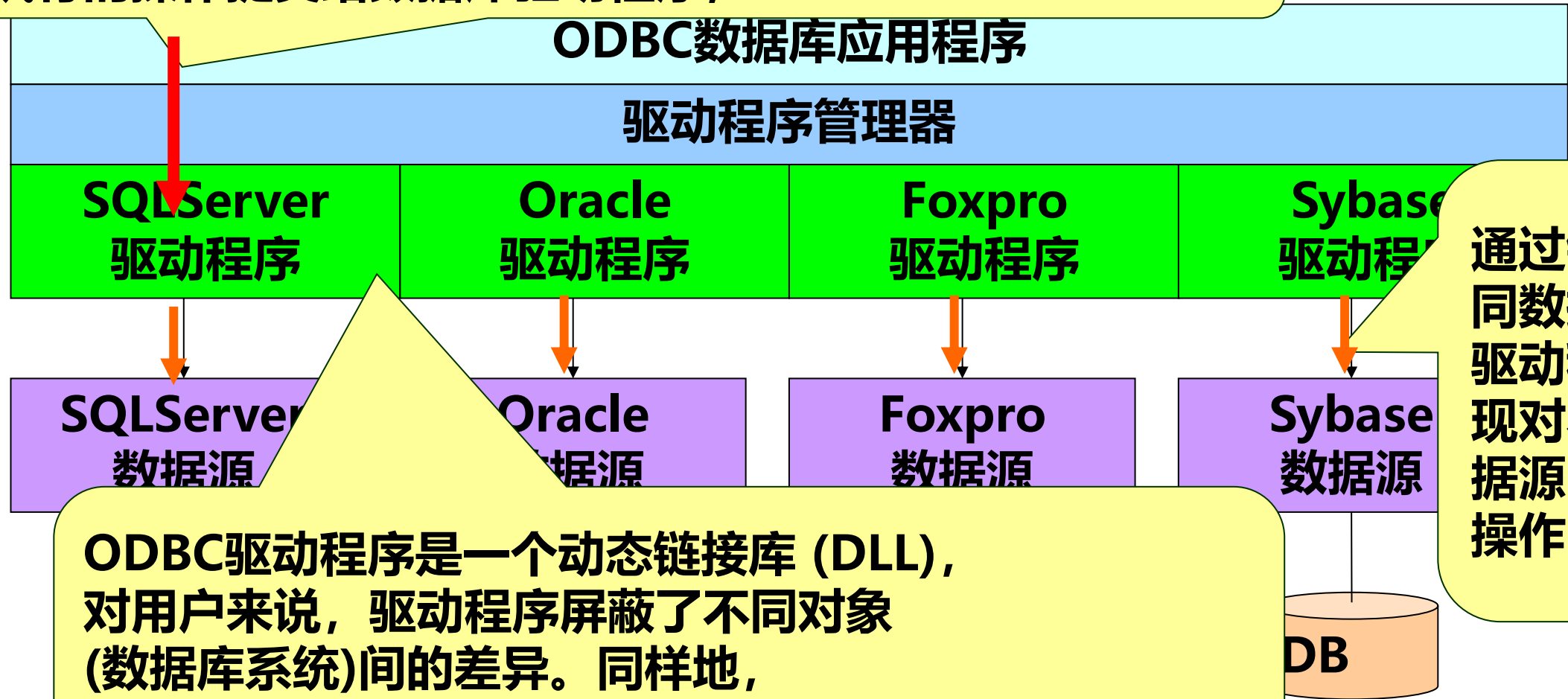
1.2 数据库访问技术

数据访问方式的变迁



1.2 数据库访问技术

ODBC应用程序不能直接存取数据库，它将所要执行的操作提交给数据库驱动程序，



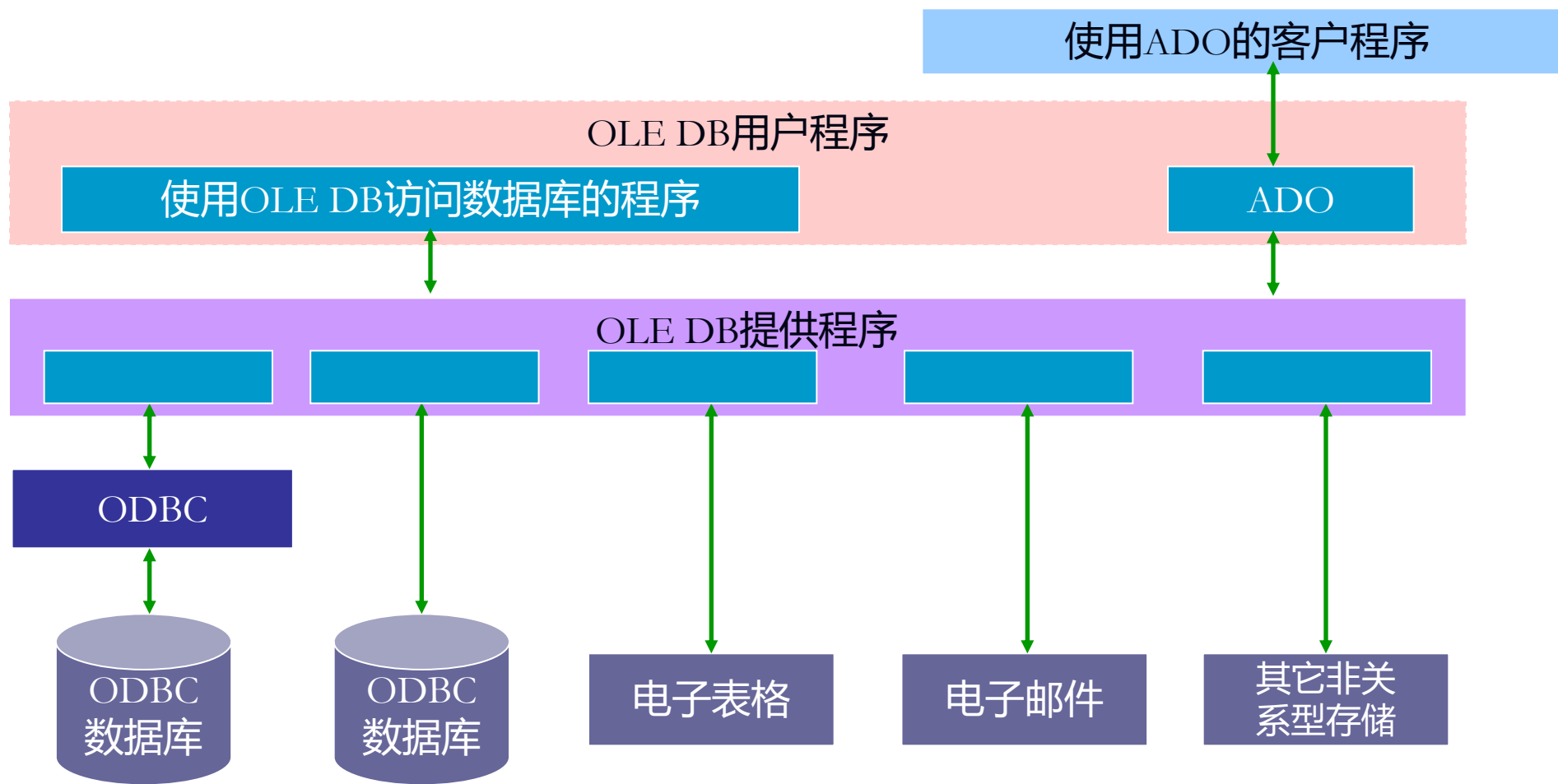
通过针对不同数据源的驱动程序实现对不同数据源的各种操作

ODBC驱动程序是一个动态链接库 (DLL)，对用户来说，驱动程序屏蔽了不同对象 (数据库系统)间的差异。同样地，ODBC屏蔽了DBMS之间的差异。

1.2 数据库访问技术

数据访问方式的变迁

• 3. OLE DB(对象链接与嵌入数据库)



1.2 数据库访问技术

数据访问方式的变迁

- 4. ADO (ActiveX Data Object, ActiveX数据对象)

建立在OLE DB之上。ADO是一个OLE DB用户程序。使用ADO的应用程序都要间接地使用OLE DB。**ADO简化了OLE DB，提供了对自动化的支持**，使得像VBScript这样的脚本语言也能够使用ADO访问数据库。

1.2 数据库访问技术

数据访问方式的变迁

- 5. ADO.net

ADO.NET是由.NET Framework为与数据库中的数据进行交互而提供的一组对象类的名称，是对Microsoft ActiveX Data Objects (ADO)一个跨时代的改进，它提供了**平台互用性和可伸缩**的数据访问。由于传送的数据都是**XML**格式的，因此任何能够读取XML格式的应用程序都可以进行数据处理。

1.2 数据库访问技术

数据访问方式的变迁



在许多应用程序中，用户访问数据后连接即关闭，在用户再次访问数据库时连接再重新打开

1.3 数据库开发实例

■ 开发需求：

某图书馆是一所大学的图书馆，馆藏各类图书200万册，期刊3000余种。读者主要对象主要是本校教师及学生，读者数约3万人，图书馆工作人员约100人，目前已经购买了计算机若干台，但尚未建立统一的集成管理系统，大部分业务工作仍靠手工完成。为了提高图书馆的工作效率和水平，更好地为读者服务，决定开发图书馆自动化系统。

■ 系统需求分析

部门结构
业务流程

1.3 数据库开发实例

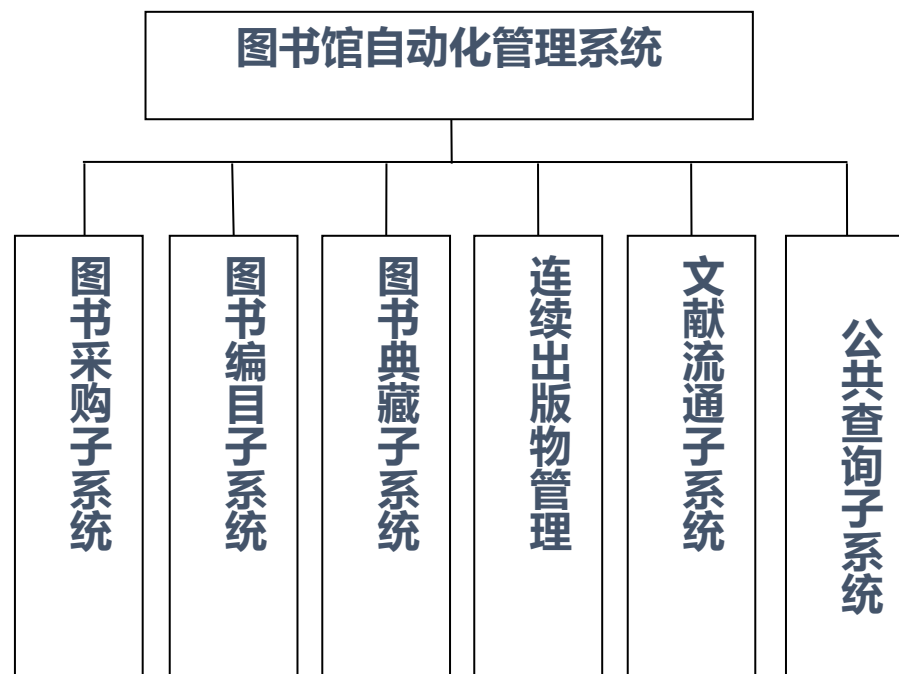
❖ 概要设计

- 概要设计是在需求分析的基础上，对系统进行基本设计，设计系统的运行环境，基本概念及处理流程，解决实现该系统的程序模块设计问题，包括如何把系统分为若干模块，决定各模块之间的接口，数据结构、运行控制、出错处理等。

❖ 数据库设计

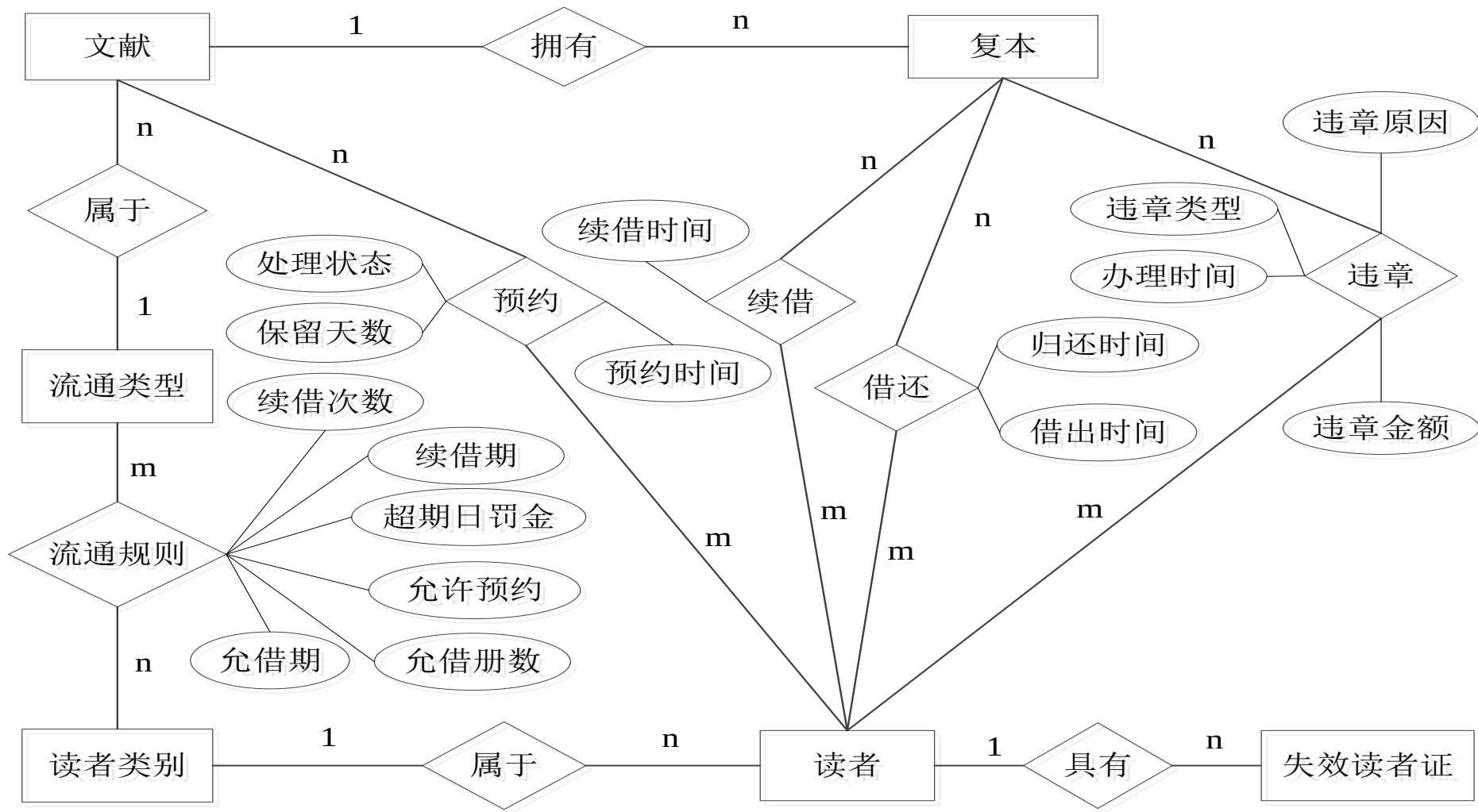
- 1) 数据库概念结构设计
 - (1) E-R图向关系模型的转化
 - (2) 关系模型的调整及优化
 - (3) 外模式设计
- 2) 逻辑结构设计
- 3) 物理结构设计

❖ 详细设计



总体功能图

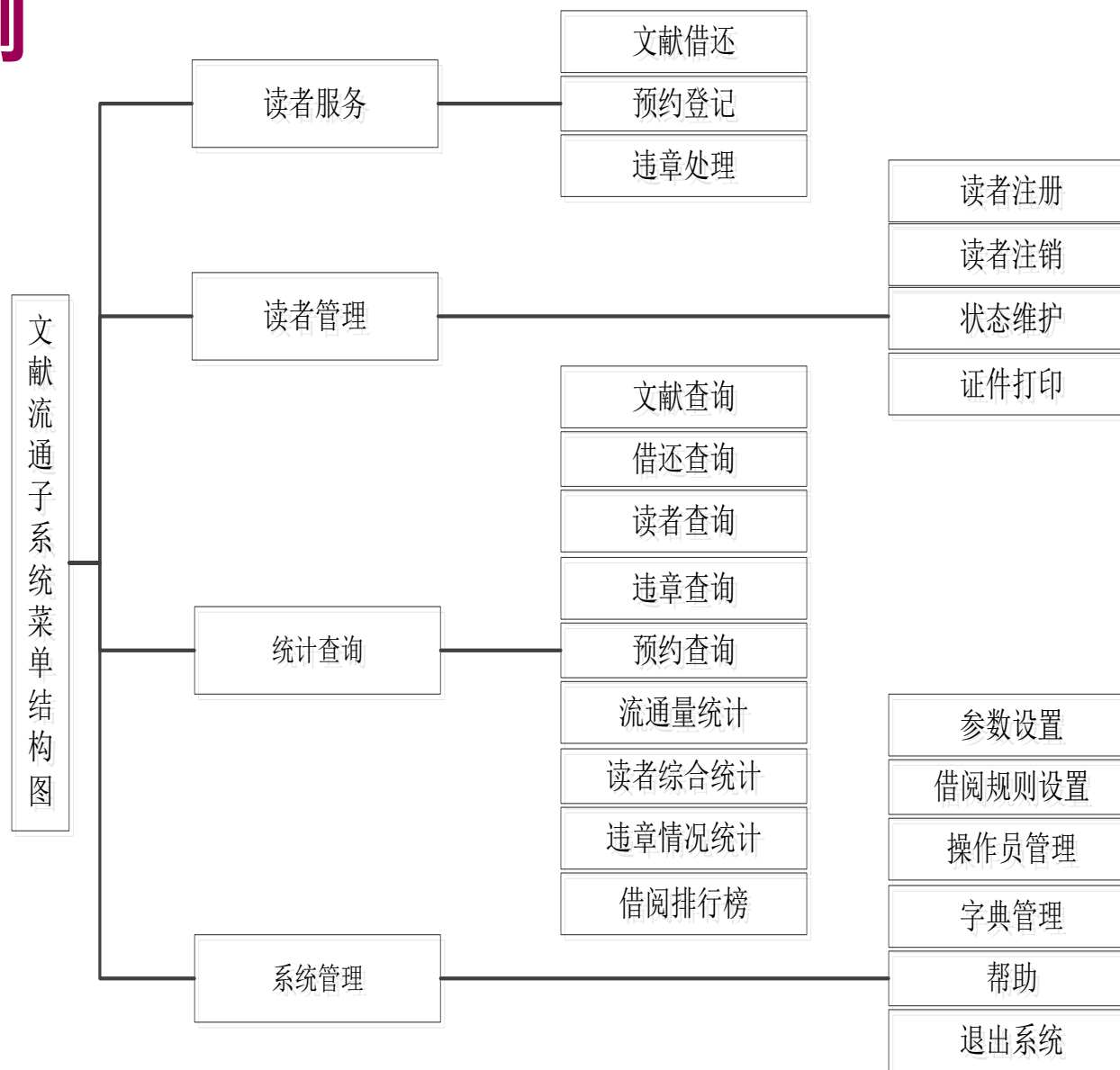
1.3 数据库开发实例



流通业务相关主要实体关系图

1.3 数据库开发实例

1. 系统功能
2. 系统构架
3. 开发工具及语言
4. 公用类库
 - 1) 基础公共类
 - 2) 项目公共类
5. 主窗体
6. 具体功能实现



菜单结构图

目录

CONTENTS

2

嵌入式SQL

2.1 嵌入式SQL语言概述

2.2 变量声明与数据库连接

2.3 数据集与游标

2.1 嵌入式SQL语言概述

交互式SQL语言的局限

交互式SQL语言有很多优点

- ❑ 记录集合操作
 - ❑ 非过程性操作：指出要做什么，而不需指出怎样做
 - ❑ 一条语句就可实现复杂查询的结果
- 然而，交互式SQL本身也有很多局限... ..

```
Select S#, Avg(Score) From SC
Where S# in
( Select S# From SC
Where Score < 60
Group by S# Having Count(*)>2 )
Group by S# ;
```

2.1 嵌入式SQL语言概述

从SQL本身角度

- 特别复杂的检索结果难以用一条交互式SQL语句完成，此时需要结合高级语言中经常出现的顺序、分支和循环结构来帮助处理
- 例如：依据不同条件执行不同的检索操作等

If some-condition Then

SQL-Query1

Else

SQL-Query2

End If

- 再如：循环地进行检索操作

Do While some-condition

SQL-Query

End Do



2.1 嵌入式SQL语言概述

高级语言+SQL语言

- ❑ 既继承高级语言的过程控制性
- ❑ 又结合SQL语言的复杂结果集操作的非过程性
- ❑ 同时又为数据库操作者提供安全可靠的操作方式：通过应用程序进行操作

嵌入式SQL语言

- ❑ 将SQL语言嵌入到某一种高级语言中使用
- ❑ 这种高级语言，如C/C++, Java, PowerBuilder等，又称**宿主语言(Host Language)**
- ❑ 嵌入在宿主语言中的SQL与前面介绍的交互式SQL有一些不同的操作方式

2.1 嵌入式SQL语言概述

示例：交互式SQL语言

```
select Sname, Sage from Student where Sname='张三' ;
```

示例：嵌入式SQL语言

□ 以宿主语言C语言为例

```
exec sql select Sname, Sage into :vSname, :vSage from Student  
where Sname='张三' ;
```

□ 典型特点

---- **exec sql**引导SQL语句: 提供给C编译器，以便对SQL语句预编译成C编译器可识别的语句

---- 增加一**into**子句: 该子句用于指出接收SQL语句检索结果的程序变量

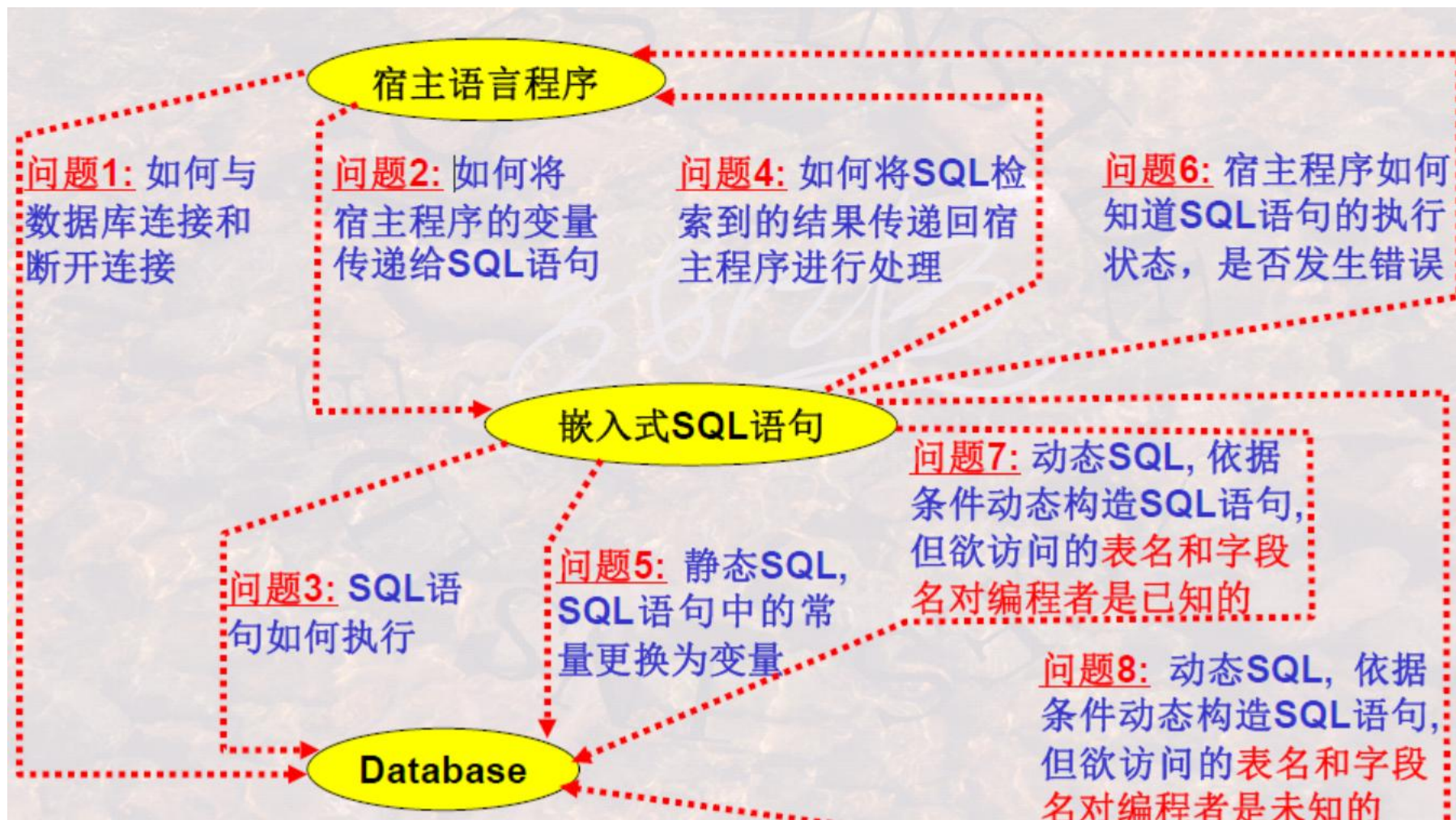
---- 由冒号引导的程序变量,如: ‘:vSname’, ‘:vSage’

□ 嵌入式SQL还有很多特点，后面将一一介绍。

2.1 嵌入式SQL语言概述

高级语言中使用嵌入式SQL语言需要解决的问题

高级语言(语句)→嵌入式SQL语句→DBMS←→DB



2.2 变量声明与数据库连接

变量的声明与使用

在嵌入式SQL语句中可以出现宿主语言语句所使用的变量：

```
exec sql select Sname, Sage into :vSname, :vSage from
```

```
Student where Sname= :specName;
```

➤ 这些变量需要特殊的声明：

```
exec sql begin declare section;
```

```
char vSname[10], specName[10]="张三" ;
```

```
int vSage;
```

```
exec sql end declare section;
```

➤ 变量声明和赋值中，要注意：

❑ 宿主程序的字符串变量长度应比字符型字段的长度多1个。因宿主程序的字符串尾部多一个终止符为 '\0'，而程序中用双引号来描述。

❑ 宿主程序变量类型与数据库字段类型之间有些是有差异的，有些DBMS可支持自动转换，有些不能。

2.2 变量声明与数据库连接

声明的变量，可以在宿主程序中赋值，然后传递给SQL语句的where等子句中，以使SQL语句能够按照指定的要求(可变化的)进行检索。

```
exec sql begin declare section;
```

```
char vSname[10], specName[10]="张三" ;
```

```
int vSage;
```

```
exec sql end declare section;
```

//用户可在此处基于键盘输入给specName赋值

```
exec sql select Sname, Sage into :vSname, :vSage from
```

```
Student where Sname = :specName ;
```

➤比较相应的交互式SQL语句：

```
select Sname, Sage from Student where Sname = '张三' ;
```

➤嵌入式比交互式SQL语句灵活了一些：只需改一下变量值，SQL语句便可反复使用，以检索出不同结果。

2.2 变量声明与数据库连接

程序与数据库的连接和断开

- 在嵌入式SQL程序执行之前，首先要与数据库进行连接
- 不同DBMS，具体连接语句的语法略有差异
- SQL标准中建议的连接语法为：

```
exec sql connect to target-server as connect-name user user-name;
```

或

```
exec sql connect to default;
```

- Oracle中数据库连接:

```
exec sql connect :user_name identified by :user_pwd;
```

- DB2 UDB中数据库连接:

```
exec sql connect to mydb user :user_name using :user_pwd;
```

2.2 变量声明与数据库连接

程序与数据库的连接和断开

在嵌入式SQL程序执行之后，需要与数据库断开连接

➤ SQL标准中建议的断开连接的语法为：

exec sql disconnect connect-name;

或

exec sql disconnect current;

➤ Oracle中断开连接：

exec sql commit release;

或

exec sql rollback release;

➤ DB2 UDB中断开连接：

exec sql connect reset;

exec sql disconnect current;

2.2 变量声明与数据库连接

SQL执行的提交与撤消

SQL语句在执行过程中，必须有**提交**和**撤消**语句才能确认其操作结果

➤SQL执行的**提交**：

```
exec sql commit work;
```

➤SQL执行的**撤消**：

```
exec sql rollback work;
```

➤为此，很多DBMS都设计了捆绑提交/撤消与断开连接在一起的语句,以保证在断开连接之前使用户确认提交或撤消先前的工作，例如Oracle中：

```
exec sql commit release;
```

或

```
exec sql rollback release;
```

2.2 变量声明与数据库连接

事务的概念与特性

事务：(从应用程序员角度)是一个存取或改变数据库内容的程序的一次执行，或者说一条或多条SQL语句的一次执行被看作一个事务

➤事务一般是由应用程序员提出，因此有开始和结束，结束前需要提交或撤消。

Begin Transaction

Exec sql ...

...

Exec sql ...

Exec sql **commit work** | exec sql **rollback work**

End Transaction

➤在嵌入式SQL程序中，任何一条数据库操纵语句(如exec sql select等)都会引发一个新事务的开始，只要该程序当前没有正在处理的事务。而事务的结束是需要应用程序员通过commit或rollback确认的。因此Begin Transaction和End Transaction两行语句是不需要的。

2.2 变量声明与数据库连接

事务的概念与特性

事务：(从微观角度，或者从**DBMS**角度)是数据库管理系统提供的控制数据操作的一种手段，通过这一手段，应用程序员将一系列的数据库操作组合在一起作为一个整体进行操作和控制，以便数据库管理系统能够提供一致性状态转换的保证。

示例：“银行转帐”事务**T**：从帐户**A**过户**5000RMB**到帐户**B**上

```
T:  read(A);  
    A := A - 5000;  
    write(A);  
    read(B);  
    B := B + 5000;  
    write(B);
```

注：**read(X)**是从数据库传送数据项**X**到事务的工作区中；**write(X)**是从事务的工作区中将数据项**X**写回数据库。

2.2 变量声明与数据库连接

事务的概念与特性

事务的特性: ACID

- ❑ **原子性 Atomicity**: DBMS 能够保证事务的一组更新操作是原子不可分的, 即对 **DB** 而言, 要么全做, 要么全不做
 - ❑ **一致性 Consistency**: DBMS 保证事务的操作状态是正确的, 符合一致性的操作规则, 它是进一步由隔离性来保证的
 - ❑ **隔离性 Isolation**: DBMS 保证并发执行的多个事务之间互相不受影响。例如两个事务 **T1** 和 **T2**, 即使并发执行, 也相当于或者先执行了 **T1**, 再执行 **T2**; 或者先执行了 **T2**, 再执行 **T1**。
 - ❑ **持久性 Durability**: DBMS 保证已提交事务的影响是持久的, 被撤销事务的影响是可恢复的。
- 换句话说: 具有 **ACID** 特性的若干数据库基本操作的组合体被称为事务。

2.2变量声明与数据库连接

示例

```
#include <stdio.h>
#include "prompt.h"
exec sql include sqlca;
char cid_prompt[ ] = "Please enter customer id: ";
int main()
{ exec sql begin declare section;
char cust_id[5], cust_name[14];
float cust_discnt;
char user_name[20],user_pwd[20];
exec sql end declare section;
exec sql whenever sqlerror goto report_error;
exec sql whenever not found goto notfound;
strcpy(user_name, "poneilsql");
strcpy(user_pwd, "XXXX");
exec sql connect :user_name identified
by :user_pwd;
```

-----SQLCA: SQL Communication Area

-----The Declare Section

-----SQL错误捕获语句/ 后

-----SQL Connect

2.2 变量声明与数据库连接

```
while((prompt(cid_prompt,1,cust_id,4)) >=0) {  
    exec sql select cname,discnt  
    into :cust_name, :cust_discnt  
    from customers where cid=:cust_id;  
    exec sql commit work;  
    printf("Customer's name is %s and discount  
    is %5.1f\n",cust_name,cust_discnt);  
    continue;  
    notfound: printf("Can't find customer %s,  
    continuing\n",cust_id); }  
    exec sql commit release;  
    return 0;  
    report_error:  
    print_dberror();  
    exec sql rollback release;  
    return 1;  
}
```

-----SQL Commit Work

-----SQL Commit Work and Disconnect

-----SQL Rollback Work and Disconnect

2.3数据集与游标

如何读取单行数据和多行数据

单行结果处理与多行结果处理的差异(Into子句与游标(Cursor))

■检索单行结果，可将结果直接传送到宿主程序的变量中

```
EXEC SQL SELECT [ALL | DISTINCT] expression [, expression...]  
INTO host-variable , [host-variable, ...]  
FROM tableref [corr_name] [ , tableref [corr_name] ...]  
WHERE search_condition;
```

示例

```
exec sql select Sname, Sage into :vSname, :vSage from Student where  
Sname = :specName ;
```

2.3数据集与游标

如何读取单行数据和多行数据

检索多行结果，则需使用游标(Cursor)

□游标是指向某检索记录集的指针

□通过这个指针的移动，每次读一行，处理一行，再读一行...，直至处理完毕

Student		
Sno	Sname	Sclass
2003510101	张三	035101
2003510102	李四	035101
2003510103	王五	035101
2003510104	李六	035101
2003510105	张四	035101
2003510106	张五	035101
2003510107	张小三	035101
2003510108	张小四	035101
2003510109	李小三	035101
2003510110	李小四	035101
2003520201	周三	035202
2003520202	赵四	035202
2003520203	赵五	035202
2003520204	赵六	035202
2003520205	钱四	035202
2003520206	强五	035202
2003520207	梁小三	035202
2003520208	梁小四	035202
2003520209	王小三	035202
2003520210	王小四	035202

DataBase



2.3数据集与游标

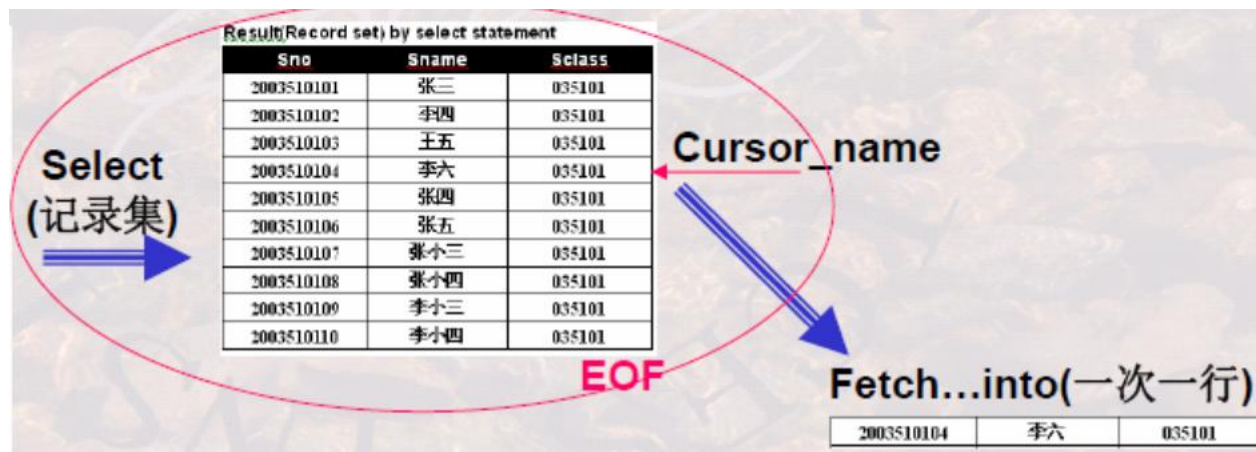
如何读取单行数据和多行数据

检索多行结果，则需使用游标(Cursor)

- 读一行操作是通过Fetch...into语句实现的：每一次Fetch,都是先向下移动指针，然后再读取
- 记录集有结束标识EOF, 用来标记后面已没有记录了

Student		
Sno	Sname	Sclass
2003510101	张三	035101
2003510102	李四	035101
2003510103	王五	035101
2003510104	李六	035101
2003510105	张四	035101
2003510106	张五	035101
2003510107	张小三	035101
2003510108	张小四	035101
2003510109	李三三	035101
2003510110	李小四	035101
2003520201	周三	035202
2003520202	赵四	035202
2003520203	赵五	035202
2003520204	赵六	035202
2003520205	钱四	035202
2003520206	强五	035202
2003520207	梁小三	035202
2003520208	梁小四	035202
2003520209	王小三	035202
2003520210	王小四	035202

DataBase



2.3数据集与游标

游标(Cursor)的使用

游标(Cursor)的使用需要先定义、再打开(执行)、接着一条接一条处理, 最后再关闭

```
exec sql declare cur_student cursor for  
select Sno, Sname, Sclass from Student where Sclass='035101';  
exec sql open cur_student;  
exec sql fetch cur_student into :vSno, :vSname, :vSclass;  
.....  
exec sql close cur_student;
```

➤游标可以定义一次, 多次打开(多次执行), 多次关闭

2.3数据集与游标

示例

```
#define TRUE 1
#include <stdio.h>
#include "prompt.h"
exec sql include sqlca;
exec sql begin declare section;
char cust_id[5], agent_id[14];
double dollar_sum;
exec sql end declare section;
int main()
{ char cid_prompt[ ]="Please enter customer ID:";
exec sql declare agent_dollars cursor for select
aid,sum(dollars)
from orders where cid = :cust_id group by aid;
exec sql whenever sqlerror goto report_error;
exec sql connect to testdb;
exec sql whenever not found goto finish;
```

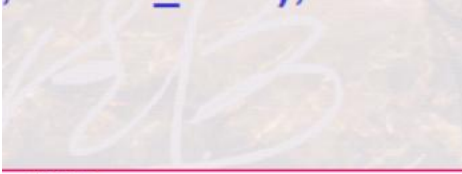
Orders				
cid	aid	product	...	dollars
C1	A1	P1		100.00
C1	A1	P2		200.00
C1	A1	P3		400.00
C1	A2	P2		100.00
C1	A2	P3		200.00
C1	A2	P1		300.00
C1	A3	P3		100.00
C1	A3	P1		200.00
C1	A3	P2		300.00
C1	A3	P1		400.00
C2	A1	P1		100.00
C2	A1	P2		200.00
C2	A1	P3		400.00
C2	A2	P2		100.00
C2	A2	P3		200.00
C2	A2	P1		300.00
C2	A2	P2		400.00
C2	A3	P3		100.00
C2	A3	P2		300.00
C2	A3	P1		400.00

agent_dollars with cid=' c1'		
Aid	dollars	
A1	700.00	
A2	600.00	
A3	1000.00	

agent_dollars with cid=' c2'		
aid	dollars	
A1	700.00	
A2	1000.00	
A3	800.00	

2.3数据集与游标

```
while((prompt(cid_prompt,1,cust_id,4)) >=0) {
exec sql open agent_dollars;
while(TRUE) {
exec sql fetch agent_dollars
into :agent_id, :dollar_sum;
printf("%s %11.2f\n",agent_id, dollar_sum);
}
finish: exec sql close agent_dollars;
exec sql commit work; }
exec sql disconnect current;
return 0;
report_error:
print_dberror();
exec sql rollback;
exec sql disconnect current;
return 1;
}
```



Orders				
cid	aid	product	...	dollars
C1	A1	P1		100.00
C1	A1	P2		200.00
C1	A1	P3		400.00
C1	A2	P2		100.00
C1	A2	P3		200.00
C1	A2	P1		300.00
C1	A3	P3		100.00
C1	A3	P1		200.00
C1	A3	P2		300.00
C1	A3	P1		400.00
C2	A1	P1		100.00
C2	A1	P2		200.00
C2	A1	P3		400.00
C2	A2	P2		100.00
C2	A2	P3		200.00
C2	A2	P1		300.00
C2	A2	P2		400.00
C2	A3	P3		100.00
C2	A3	P2		300.00
C2	A3	P1		400.00

agent_dollars with cid=' c1'	
Aid	dollars
A1	700.00
A2	600.00
A3	1000.00

agent_dollars with cid=' c2'	
aid	dollars
A1	700.00
A2	1000.00
A3	800.00

2.3数据集与游标

游标的使用方法

Cursor的定义： declare cursor

EXEC SQL DECLARE cursor_name CURSOR FOR

Subquery

[ORDER BY result_column [ASC | DESC][, result_column ...]

[FOR [READ ONLY | UPDATE [OF columnname [, columnname...]]]]];

示例

exec sql declare **cur_student** cursor for

select **Sno, Sname, Sclass** from **Student** where **Sclass= :vClass**

order by **Sno**

for read only ;

Cursor的打开和关闭： open cursor //close cursor

EXEC SQL OPEN cursor_name;

EXEC SQL CLOSE cursor_name;

2.3数据集与游标

游标的使用方法

Cursor的数据读取: Fetch

```
EXEC SQL FETCH cursor_name  
INTO host-variable , [host-variable, ...];
```

示例

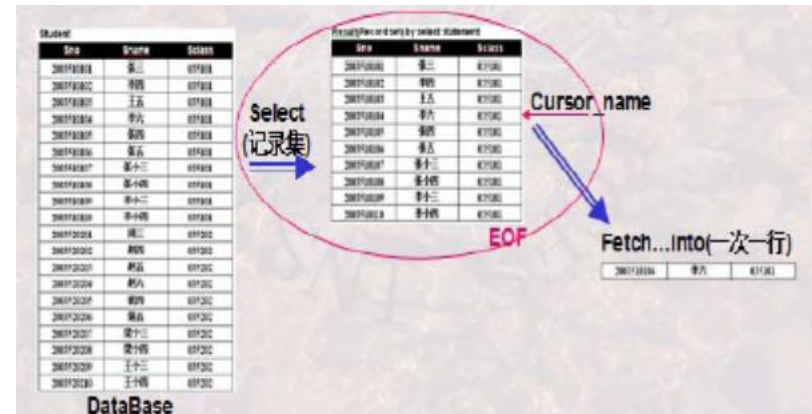
```
exec sql declare cur_student cursor for  
select Sno, Sname, Sclass from Student where  
Sclass= :vClass  
order by Sno for read only ;  
exec sql open cur_student;  
...  
exec sql fetch cur_student  
into :vSno, :vSname, :vSage  
...  
exec sql close cur_student;
```

2.4可滚动游标及数据库的增删改

ODBC支持的可滚动Cursor

➤标准的游标始终是自开始向结束方向移动的，每fetch一次，向结束方向移动一次；一条记录只能被访问一次；再次访问该记录只能关闭游标后重新打开

```
exec sql declare cur_student cursor for
select Sno, Sname, Sclass from Student where Sclass= :vClass
order by Sno for read only ;
exec sql open cur_student;
...
exec sql fetch cur_student
into :vSno, :vSname, :vSage ;
...
exec sql close cur_student;
```



➤ODBC(Open DataBase Connectivity)是一种跨DBMS的DB操作平台，它在应用程序与实际的DBMS之间提供了一种通用接口➤许多实际的DBMS并不支持可滚动游标，但通过ODBC可以使用该功能

2.4可滚动游标及数据库的增删改

可滚动游标的定义和使用

可滚动游标是可使游标指针在记录集之间灵活移动、使每条记录可以反复被访问的一种游标

```
EXEC SQL DECLARE cursor_name [INSENSITIVE] [SCROLL] CURSOR  
[WITH HOLD] FOR Subquery  
[ORDER BY result_column [ASC | DESC][, result_column ...]  
[FOR READ ONLY | FOR UPDATE OF columnname [,  
columnname ]...];  
EXEC SQL FETCH  
[ NEXT | PRIOR | FIRST | LAST  
| [ABSOLUTE | RELATIVE] value_spec ]  
FROM cursor_name INTO host-variable [, host-variable ...];
```

➤NEXT向结束方向移动一条； PRIOR向开始方向移动一条； FIRST回到第一条； LAST移动到最后一条； ABSOLUT value_spec定向检索指定位置的行, value_spec由1至当前记录集最大值； RELATIVE value_spec相对当前记录向前或向后移动, value_spec为正数向结束方向移动, 为负数向开始方向移动

2.4可滚动游标及数据库的增删改

可滚动游标的定义和使用

可滚动游标移动时需判断是否到结束位置，或到起始位置

□可通过判断是否到EOF位置(最后一条记录的后面)，或BOF位置(起始记录的前面)

□如果不需区分，可通过whenver not found语句设置来检测



2.4可滚动游标及数据库的增删改

数据库记录的删除

➤一种是查找删除(与交互式**DELETE**语句相同)，一种是定位删除

```
EXEC SQL DELETE FROM tablename [corr_name]  
WHERE search_condition | WHERE CURRENT OF cursor_name;
```

示例：查找删除

```
exec sql delete from customers c where c.city = 'Harbin' and  
not exists ( select * from orders o where o.cid = c.cid);
```

示例：定位删除

```
exec sql declare delcust cursor for  
select cid from customers c where c.city='harbin' and  
not exists ( select * from orders o where o.cid = c.cid)  
for update of cid;  
exec sql open delcust  
While (TRUE) {  
exec sql fetch delcust into :cust_id;  
exec sql delete from customers where current of delcust ; }
```


2.4可滚动游标及数据库的增删改

数据库记录的更新

➤一种是查找更新(与交互式Update语句相同)，一种是定位更新

```
EXEC SQL UPDATE tablename [corr_name]
SET columnname = expr [, columnname = expr ...]
[ WHERE search_condition ] | WHERE CURRENT OF cursor_name;
```

示例：查找更新

```
exec sql update student s set sclass = '035102'
where s.sclass = '034101'
```

示例：定位更新

```
exec sql declare stud cursor for
select * from student s where s.sclass ='034101'
for update of sclass;
exec sql open stud
While (TRUE) {
exec sql fetch stud into :vSno, :vSname, :vSclass;
exec sql update student set sclass = '035102' where current of
stud ; }
```


2.4可滚动游标及数据库的增删改

数据库记录的插入

只有一种类型的插入语句

```
EXEC SQL INSERT INTO tablename [ (columnname [,columnname, ...] )]  
[ VALUES (expr [ , expr , ...] ) | subquery ] ;
```

示例：插入语句

```
exec sql insert into student ( sno, sname, sclass)  
values ('03510128', '张三' , '035101') ;
```

示例：插入语句

```
exec sql insert into masterstudent ( sno, sname, sclass)  
select sno, sname, sclass from student;
```

2.4可滚动游标及数据库的增删改

**示例：宿主语言与SQL结合的过程性控制
求数据库中某一行位于中值的那一行**

```
#include <stdio.h>
#include "prompt.h"
exec sql include sqlca;
char custprompt[ ] ="Please enter a customer ID: "
int main()
{
exec sql begin declare section;
char cid[5],user_name[20], user_pwd[10];
double dollars; int ocount;
exec sql end declare section;
exec sql declare dollars_cursor cursor for
select dollars from orders where cid = :cid and dollars
is not null
order by dollars;
int i;
```

Orders				
cid	aid	product		dollars
C1	A1	P1		100.00
C1	A2	P2		100.00
C1	A3	P3		100.00
C1	A3	P1		200.00
C1	A1	P2		200.00
C1	A2	P3		200.00
C1	A3	P2		300.00
C1	A2	P1		300.00
C1	A1	P3		400.00
C1	A3	P1		400.00
C2	A1	P1		100.00
C2	A3	P3		100.00
C2	A2	P2		100.00
C2	A2	P3		200.00
C2	A1	P2		200.00
C2	A2	P1		300.00
C2	A3	P2		300.00
C2	A3	P1		400.00
C2	A2	P2		400.00
C2	A1	P3		400.00

2.4可滚动游标及数据库的增删改

```

exec sql whenever sqlerror goto report_error;
strcpy(user_name, "poneilsql");
strcpy(user_pwd, "xxxx");
exec sql connect :user_name identified by :user_pwd;
While ( prompt(custprompt, 1, cid ,4)>=0 {
exec sql select count(dollars) into :ocount from
orders where cid = :cid ;
if (ocount ==0 ){
printf("No record reviewed for cid value %s\n", cid);
continue; }
exec sql open dollars_cursor;
for (i =0; i< (ocount+1)/2; i++)
exec sql fetch dollars_cursor into :dollars;
exec sql close dollars_cursor;
exec sql commit work;
printf("Median dollar amount =%f\n", dollars); }
...}

```

Orders

cid	aid	product		dollars
C1	A1	P1		100.00
C1	A2	P2		100.00
C1	A3	P3		100.00
C1	A3	P1		200.00
C1	A1	P2		200.00
C1	A2	P3		200.00
C1	A3	P2		300.00
C1	A2	P1		300.00
C1	A1	P3		400.00
C1	A3	P1		400.00
C2	A1	P1		100.00
C2	A3	P3		100.00
C2	A2	P2		100.00
C2	A2	P3		200.00
C2	A1	P2		200.00
C2	A2	P1		300.00
C2	A3	P2		300.00
C2	A3	P1		400.00
C2	A2	P2		400.00
C2	A1	P3		400.00

2.5 状态捕获及错误处理机制

状态捕获及其处理

- 状态，是嵌入式SQL语句的执行状态，尤其指一些出错状态；有时程序需要知道这些状态并对这些状态进行处理
- 嵌入式SQL程序中，状态捕获及处理有三部分构成

□设置SQL通信区：一般在嵌入式SQL程序的开始处便设置

exec sql include sqlca;

□设置状态捕获语句：在嵌入式SQL程序的任何位置都可设置；可多次设置；但有作用域

exec sql whenever sqlerror goto report_error;

□状态处理语句：某一段程序以应对SQL操作的某种状态

report_error: exec sql rollback;

2.5 状态捕获及错误处理机制

状态捕获语句

`exec sql whenever condition action;`

➤ **Whenever**语句的作用是设置一个“条件陷阱”，该条语句会对其后面的所有由**Exec SQL**语句所引起的对数据库系统的调用自动检查它是否满足条件(由**condition**指出)。

❑ **SQLERROR**: 检测是否有**SQL**语句出错。其具体意义依赖于特定的**DBMS**

❑ **NOT FOUND**: 执行某一**SQL**语句后，没有相应的结果记录出现

❑ **SQLWARNING**: 不是错误，但应引起注意的条件

➤ 如果满足**condition**，则要采取一些动作(由**action**指出)

❑ **CONTINUE**: 忽略条件或错误，继续执行

❑ **GOTO 标号**: 转移到标号所指示的语句，去进行相应的处理

❑ **STOP**: 终止程序运行、撤消当前的工作、断开数据库的连接

❑ **DO**函数或**CALL**函数: 调用宿主程序的函数进行处理，函数返回后从引发该**condition**的**Exec SQL**语句之后的语句继续进行

2.5 状态捕获及错误处理机制

典型DBMS系统记录状态信息的三种方法

状态记录

❑ **sqlcode**: 典型DBMS都提供一个**sqlcode**变量来记录其执行**sql**语句的状态，但不同DBMS定义的**sqlcode**值所代表的状态意义可能是不同的，需要查阅相关的DBMS资料来获取其含义。

✓ **sqlcode == 0**, successful call;

✓ **sqlcode < 0**, error, e.g., from connect, database does not exist , -16;

✓ **sqlcode > 0**, warning, e.g., no rows retrieved from fetch

❑ **sqlca.sqlcode**: 支持SQLCA的产品一般要在SQLCA中填写**sqlcode**来记录上述信息；除此而外，**sqlca**还有其他状态信息的记录

❑ **sqlstate**: 有些DBMS提供的记录状态信息的变量是**sqlstate**或**sqlca.sqlstate**

➤ 当我们不需明确知道错误类型，而只需知道发生错误与否，则我们只要使用前述的状态捕获语句即可，而无需关心状态记录变量(隐式状态处理)

➤ 但我们程序中如要自行处理不同状态信息时，则需要知道以上信息，但也需知道正确的操作方法(显式状态处理)

目录

CONTENTS

3

动态SQL

3.1 动态SQL的概念和作用

3.2 SQL语句的动态构造

3.3 动态SQL语句的执行方式

3.1 动态SQL的概念和作用

静态SQL

◆概念：SQL语句中主变量的个数与数据类型在预编译时都是确定的，只有是主变量的值是程序运行过程中动态输入的。也称为嵌入式SQL语句。

◆优点：用户可以在程序运行过程中根据实际需要输入WHERE子句或HAVING子句中某些变量的值。

◆缺点：提供的编程灵活性在许多情况下显得不足，不能编写更为通用的程序。

```
SpecName = '张三' ;  
exec sql select Sno, Sname, Sclass  
into :vSno, :vSname, :vSclass from  
Student where Sname= :SpecName ;
```

或

```
exec sql declare cur_student cursor for  
select Sno, Sname, Sclass from Student where  
Sclass= :vClass  
order by Sno for read only ; /*定义*/
```

```
exec sql open cur_student ; /*执行*/
```

SQL语句在程序中已经按要求写好，只需要把一些参数通过变量(高级语言程序语句中**不带冒号**) 传送给嵌入式SQL语句即可(嵌入式SQL语句中**带冒号**)

3.1 动态SQL的概念和作用

动态SQL

```
#include <stdio.h>
exec sql include sqlca;
exec sql begin declare section;
char user_name[ ] = "Scott"; char user_pwd[ ] = "tiger";
char sqltext[ ] = " delete from customers where cid = 'c006' ";
exec sql end declare section;
int main()
{
exec sql whenever sqlerror goto report_error;
exec sql connect :user_name identified by :user_pwd;
exec sql execute immediate :sqltext;
exec sql commit release; return 0;
report_error: print_dberror(); exec sql rollback release; return 1;
}
```

动态SQL**特点**：SQL语句可以在程序中动态构造，形成一个字符串，如上例sqltext，然后再交给DBMS执行，交给DBMS执行时仍旧可以传递变量

3.1 动态SQL的概念和作用

动态构造SQL语句是应用程序员必须掌握的重要手段

示例：编写由用户确定检索条件的应用程序

请输入 Y_N Y 姓名 张三

Y_N N 年龄在 和 之间

Y_N Y 班级 035103

... ..

用户输入检索条件

显示动态SQL语句构造结果(字符串)

显示动态SQL语句执行结果

Entitled

☒ 学号 200201% ☐ 班级
☒ 姓名 张% ☐ 系别
☒ 年龄自 18 到 23 ☐ 地址
☒ 性别 男

select * from student where (sid like '200201%') and (sname like '张%') and (ssex = '男') and (sage >= 18) and (sage <= 23)

Sid	Sname	Sage	Ssex	Sclass	Sdept	Saddr
2002010201	张二	20	男	20020102	03	吉林省长春市
2002010101	张一	20	男	20020101	03	吉林省长春市

退出

3.1 动态SQL的概念和作用

如何设计用户的操作界面?

Untitled

☒ 学号 200201% ☐ 班级
☒ 姓名 张% ☐ 系别
☒ 年龄自 18 到 23 ☐ 地址
☒ 性别 男 ☐

查询

select * from student where (sid like 200201%) and (sname like 张%) and (ssex = '男') and (sage >= 18) and (sage <= 23)

Sid	Sname	Sage	Ssex	Sclass	Sdept	Saddr
2002010201	张二	26	男	20020102	03	吉林省长春市
2002010101	张一	26	男	20020101	03	吉林省长春市

QBE : Query by Example

(See also “关系模型之关系演算”)

Student	S#	Sname	Ssex	Sage	D#	Sclass
		P.X	Male	<17		
✓		P.Y	Female	>20		

(用户)
界面操作
表单型语言

数据库应用程序

(应用程序员)

QBE

(应用程序员)

SQL

数据库管理系统实现程序

(DBMS)

SQL

(DBMS)

关系代数

3.2 SQL语句的动态构造

示例一：(1)程序背景

已知关系：Customers(Cid, Cname, City, discnt)

➤从Customers表中删除满足条件的行

Delete customer rows with ALL of the following properties:

Y_N_Y Customer name is Mbale Jamson____

Y_N_N Customer is in city _____

Y_N_N Customer discount is in range from _____ to _____

...(and so on)

➤假设：用户界面上的输入存在下面的变量中

•char•Vcname[•];

•char•Vcity[•];

•double•range_from,•range_to;

•int•Cname_chose,•City_chose,•Discnt_chose;

请按用户输入构造相应的SQL语句

3.2 SQL语句的动态构造

示例一：(2)程序构造

```
#include <stdio.h>
#include "prompt.h"
char Vcname[ ];
char Vcity[ ];
double range_from, range_to;
int Cname_chose, City_chose, Discnt_chose;
Cname_chose = 0; City_chose = 0; Discnt_chose = 0;
int sql_sign = 0;
char continue_sign[ ];

exec sql include sqlca;

exec sql begin declare section;
char user_name[20],user_pwd[20];
char sqltext[ ]="delete from customers where ";
exec sql end declare section;
```

-----程序变量声明

-----SQLCA: SQL Communication Area

----- The Declare Section

```
•char Vcname[ ];
•char Vcity[ ];
•double range_from, range_to;
•int Cname_chose, City_chose, Discnt_chose;
```

3.2 SQL语句的动态构造

```
int main()
{
    exec sql whenever sqlerror goto report_error;
    strcpy(user_name, "poneilsql");
    strcpy(user_pwd, "XXXX");
    exec sql connect :user_name identified
    by :user_pwd;
    while(1) {
        memset(Vcname, '\0', 20);
        memset(Vcity, '\0', 20);
        if (GetCname(Vcname))
            Cname_chose = 1;
        if (GetCity(Vcity))
            City_chose = 1;
        if (GetDiscntRange(&range_from, &range_to))
            Discnt_chose = 1;
```

Customers(Cid, Cname, City, discnt)

-----SQL错误捕获语句

-----SQL Connect

-----初始化

-----获取Cname值

-----获取City值

-----获取Discnt区间值

```
•char Vcname[ ];
•char Vcity[ ];
•double range_from, range_to;
•int Cname_chose, City_chose, Discnt_chose;
```


3.2 SQL语句的动态构造

```
if (Cname_chose){  
    sql_sign = 1;  
    strcat(sqltext, "Cname = '");  
    strcat(sqltext, Vcname);  
    strcat(sqltext, "'");  
}
```

-----如果选择了 **Cname**，构造 **sqltext**

```
if (City_chose){  
    sql_sign = 1;  
    if(Cname_chose)  
        strcat(sqltext, " and City = '");  
    else  
        strcat(sqltext, " City = '");  
    strcat(sqltext, Vcity);  
    strcat(sqltext, "'");  
}
```

-----如果选择了 **City**，构造 **sqltext**

Customers(Cid, Cname, City, discnt)

```
•char Vcname[ ];  
•char Vcity[ ];  
•double range_from, range_to;  
•int Cname_chose, City_chose, Discnt_chose;
```

3.2 SQL语句的动态构造

```
if (Discnt_chose){  
    sql_sign =1;  
    if(Cname_chose=0 and City_chose = 0)  
        strcat(sqltext, " discnt >");  
    else  
        strcat(sqltext, " and (discnt >");  
        strcat(sqltext, dtoa(range_from));  
        strcat(sqltext, " and discnt<");  
        strcat(sqltext, dtoa(range_to));  
        strcat(sqltext, ")");  
}
```

```
if(sql_sign){  
    exec sql execute immediate :sqltext;  
    exec sql commit work;  
}
```

Customers(Cid, Cname, City, discnt)

-----如果选择了 **Discnt** 区间值，构造 **sqltext**

-----动态 **SQL** 语句执行

-----**SQL Commit Work**

```
•char Vcname[ ];  
•char Vcity[ ];  
•double range_from, range_to;  
•int Cname_chose, City_chose, Discnt_chose;
```


3.2 SQL语句的动态构造

```
scanf("continue (y/n) %1s", continue_sign)
if(continue_sign = "n"){
exec sql commit release;
return 0;
}
} -----while 结束
```

```
report_error:
print_dberror();
exec sql rollback release;
return 1;
} -----main 结束
```

-----SQL Commit Work and Disconnect

-----SQL Rollback Work and Disconnect

Customers(Cid, Cname, City, discnt)

```
•char Vcname[ ];
•char Vcity[ ];
•double range_from, range_to;
•int Cname_chose, City_chose, Discnt_chose;
```

3.2 SQL语句的动态构造

示例一：动态SQL语句构造小结

```
char sqltext[] = "delete from customers where " ;
```

```
...
```

```
if (Cname_chose) { sql_sign = 1; strcat(sqltext, "Cname = '");
```

```
    strcat(sqltext, Vcname); strcat(sqltext, "'"); }
```

```
if (City_chose) { sql_sign = 1;
```

```
    if (Cname_chose)
```

```
        strcat(sqltext, " and City = '");
```

```
    else
```

```
        strcat(sqltext, " City = '");
```

```
        strcat(sqltext, Vcity);
```

```
        strcat(sqltext, "'"); }
```

```
if (Discnt_chose) {
```

```
    sql_sign = 1;
```

```
    if (Cname_chose=0 and City_chose = 0)
```

```
        strcat(sqltext, " discnt >");
```

```
    else
```

```
        strcat(sqltext, " and (discnt >");
```

```
        strcat(sqltext, dtoa(range_from));
```

```
        strcat(sqltext, " and discnt <");
```

```
        strcat(sqltext, dtoa(range_to));
```

```
        strcat(sqltext, " "); }
```

写好基本部分

Sqltext="delete from customers where "

基本语句

Sqltext=Sqltext+ "Cname= ' "+ Vcname + " ' "

字符串需加引号

Sqltext=Sqltext + "City= ' "+ Vcity + " ' "

或者 Sqltext=Sqltext + " and City= ' "+ Vcity + " ' "

添加逻辑运算符

Sqltext=Sqltext + "Discnt > " + dtoa(range_from)

数值变量转换成字符串

3.2 SQL语句的动态构造

示例二：程序背景

示例: 编写程序, 依据用户输入条件构造SQL语句并执行

用户输入检索条件

显示动态SQL语句
构造结果(字符串)

显示动态SQL
语句执行结果

Untitled

☒ 学号 200201%
 ☐ 班级
 ☐ 系别
 ☐ 地址
 ☒ 姓名 张%
 ☐ 性别 男
 ☒ 年龄自 18 到 23

select * from student where (sid like '200201%') and (sname like 张%) and (ssex = '男') and (sage >= 18) and (sage <= 23)

Sid	Sname	Sage	Ssex	Sclass	Sdept	Saddr
2002010201	张二	20	男	20020102	03	吉林省长春市
2002010101	张一	20	男	20020101	03	吉林省长春市

返回

3.2 SQL语句的动态构造

示例二：程序背景

界面要素及其背后的变量简介

cbx_id.checked **sle_id.text**
cbx_name.checked **sle_name.text**
cbx_agest.checked **sle_agest.text** **cbx_class.checked** **sle_class.text**
cbx_sex.checked **sle_ageed.text** **cbx_dept.checked** **sle_dept.text**
 sle_sex.text **cbx_addr.checked** **sle_addr.text**

sle_sqltext.text

对象. 属性

<复选框>.checked
 =true 被选中
 =false 未被选中

<文本框>.text
 键盘输入的内容
 或者待显示的内容

学号 姓名 年龄自 到 性别 班级 系别 地址 查询

Sid	姓名	年龄	性别	班级	系别	地址
2002010101	王一	20	男	20020101	03	吉林省长春市
2002010102	张二	19	男	20020102	03	黑龙江省伊春市
2002010103	李二					

返回

3.2 SQL语句的动态构造

示例二：动态SQL语句的构造

SQL字符串的构造

cbx_id.checked sle_id.text
 cbx_name.checked sle_name.text
 cbx_age.checked sle_ageed.text
 cbx_sex.checked sle_sex.text
 cbx_class.checked sle_class.text
 cbx_dept.checked sle_dept.text
 cbx_addr.checked sle_addr.text

BatchFind

广学号: 广转码:
 广姓名: 广性别:
 广年龄: 广地址:
 广性别:

查询

sle_sqltext.text

Sid	Sname	Sex	Age	Sdept	Saddr
200201001	张三	男	20	01	吉林省长春市
200201002	李一	女	20	01	吉林省吉林市
200201003	王二	男	21	01	黑龙江省哈尔滨市
200201004	张三	男	20	01	吉林省长春市
200201005	李二	男	19	01	黑龙江省哈尔滨市

返回

怎样将输入和SQL语句合并在一起?

Char型属性条件如何构造?

写好基本部分

基本语句

字符串需加引号

添加逻辑运算符

trim()去两边的空格, len()求长度

```

string str_temp
int firstflag = 0
str_temp = ""

// dw_2.setfilter("")
Str_temp = " select ' from student where "
if (cbx_id.checked = true and len(trim(sle_id.text))>0) then
    str_temp = str_temp + "(sid like '" + trim(sle_id.text) + "')"
    firstflag = 1
end if
if (cbx_name.checked = true and len(trim(sle_name.text))>0) then
    if (firstflag = 1) then
        str_temp = str_temp + " and (sname like '" + trim(sle_name.text) + "')"
    else
        str_temp = str_temp + " ( sname like '" + trim(sle_name.text) + "')"
        firstflag = 1
    end if
end if
if (cbx_sex.checked = true and len(trim(sle_sex.text))>0) then
    if (firstflag = 1) then
        str_temp = str_temp + " and ( ssex = '" + trim(sle_sex.text) + "')"
    else
        str_temp = str_temp + " (ssex = '" + trim(sle_sex.text) + "')"
        firstflag = 1
    end if
end if

```

3.2 SQL语句的动态构造

```

if (cbx_addr.checked = true and len(trim(sle_addr.text))>0) then
  if (firstflag = 1) then
    str_temp = str_temp + " and (saddr like '" + trim(sle_addr.text) + "'" )
  else
    str_temp = str_temp + " (saddr like '" + trim(sle_addr.text) + "'" )
    firstflag = 1
  end if
end if
if (cbx_agest.checked = true and len(trim(sle_agest.text))>0 ) then
  if (firstflag = 1) then
    str_temp = str_temp + " and ( sage >= " + trim(sle_agest.text) + " )"
    if (len(trim(sle_ageed.text))>0 and integer(trim(sle_ageed.text))> integer(trim(sle_agest.text))) then
      str_temp = str_temp + " and ( sage <= " + trim(sle_ageed.text) + " )"
    end if
  else
    str_temp = str_temp + " (sage >= " + trim(sle_agest.text) + " )"
    if ( len(trim(sle_ageed.text))>0 and integer(trim(sle_ageed.text))>integer(trim(sle_agest.text))) then
      str_temp = str_temp + " and (sage <= " + trim(sle_ageed.text) + " )"
    end if
  end if
  firstflag = 1
end if
end if
//sle_sqltext.text = str_temp
sqltext = str_temp
exec sql execute immediate :sqltext
exec sql commit release

// dw_2.setfilter(str_temp)
// dw_2.filter()

```

文本框中输入的是字符，
比较时要转换成数值

Integer型属性条件
如何构造？

数值变量转换成字符串

构造好的SQL字
符串如何执行？

执行构造好的SQL字符串

```

cbx_id.checked sle_id.text
cbx_name.checked sle_name.text
cbx_agest.checked sle_agest.text
cbx_ageed.checked sle_ageed.text
cbx_sex.checked sle_sex.text
cbx_class.checked sle_class.text
cbx_dept.checked sle_dept.text
cbx_addr.checked sle_addr.text

```

sle_sqltext.text

Sid	Sname	Sage	Ssex	Sdept	Saddr
20020101	张三	28	男	2002001	吉林省长春市
20020102	李一	28	女	2002001	吉林省长春市
20020103	王二	21	男	2002001	黑龙江省哈尔滨市
20020104	赵三	38	男	2002002	吉林省长春市
20020105	孙四	15	男	2002002	黑龙江省哈尔滨市

3.2 SQL语句的动态构造

示例二：动态SQL语句的执行结果

示例结果1:

☒ 学号
 200201%

☐ 班级

☒ 姓名
 张%

☐ 系别

☒ 年龄自
 18
 到
 23

☐ 地址

☒ 性别
 男

查询

```
select * from student where (sid like '200201%') and (sname like '张%') and (ssex = '男') and (sage >= 18) and (sage <= 23)
```

Sid	Sname	Sage	Ssex	Sclass	Sdept	Saddr
2002010201	张二	20	男	20020102	03	吉林省长春市
2002010101	张一	20	男	20020101	03	吉林省长春市

返回

3.2 SQL语句的动态构造

示例二：动态SQL语句的执行结果

示例结果2:

Untitled

☐ 学号
☐ 姓名
☒ 年龄自 18 到 23
☐ 性别

☒ 班级 20020101
☐ 系别
☒ 地址 吉林%

查询

```
select * from student where ( sclass like '20020101') and (saddr like '吉林%') and ( sage >= 18) and ( sage <= 23)
```

Sid	Sname	Sage	Ssex	Sclass	Sdept	Saddr
2002010101	张一	20	男	20020101	03	吉林省长春市
2002010102	李一	20	女	20020101	03	吉林省吉林市

自动生成SQL并执行

返回



3.3 动态SQL语句的执行方式

动态SQL的两种执行方式

如SQL语句已经被构造在host-variable字符串变量中,则:

□**立即执行语句**: 运行时编译并执行

EXEC SQL EXECUTE IMMEDIATE :host-variable;

□**Prepare-Execute-Using语句**: PREPARE语句先编译, 编译

后的SQL语句允许动态参数, EXECUTE语句执行, 用USING语句将动态参数值传送给编译好的SQL语句

EXEC SQL PREPARE sql_temp FROM :host-variable;

.....

EXEC SQL EXECUTE sql_temp USING :cond-variable

3.3 动态SQL语句的执行方式

Prepare-Execute-Using的例子

```
#include <stdio.h>
#include "prompt.h"
exec sql include sqlca;
exec sql begin declare section;
char cust_id[5], sqltext[256], user_name[20], user_pwd[10];
exec sql end declare section;
char cid_prompt[ ] = "Name customer cid to be deleted: ";
int main()
{
    strcpy(sqltext, "delete from customers where cid = :dcid");
    .....
    while (prompt(cid_prompt, 1, cust_id, 4) >= 0) {
        exec sql whenever not found goto no_such_cid;
        exec sql prepare delcust from :sqltext; /* prepare statement */
        exec sql execute delcust using :cust_id; /* using clause ... replaces ":n" above */
        exec sql commit work; continue;
        no_such_cid: printf("No cust %s in table\n",cust_id);
        continue;
    }
    ...
}
```

3.3 动态SQL语句的执行方式

```
... ..  
strcpy(sqltext, "delete from customers where cid = 'C01'");  
... ..  
exec sql execute immediate :sqltext;  
exec sql commit work;
```

构造的字符串
SQL内部没有
“变量” 参数

```
... ..  
strcpy(sqltext, "delete from customers where cid = :dcid");  
... ..  
exec sql prepare delcust from :sqltext; /* prepare statement */  
exec sql execute delcust using :cust_id; /* using clause ... replaces ":n" above */  
exec sql commit work;
```

构造的字符串
SQL内部有
“变量” 参数

谢谢大家!

